

Application Layer Protocols: HTTP, DNS & Email

Note: Like the other flagged topics, this isn't covered substantively in your source notes (which only name these protocols) — drafted fresh from general SDE-interview networking knowledge, in the same style as the rest.

PART 1 — HTTP (HyperText Transfer Protocol)

1. The Basics

HTTP is the Application-layer protocol underlying the web — a **request-response** protocol where a client (browser) sends a request and a server sends back a response. It runs **over TCP** (a reliable, connection-oriented transport is a deliberate choice, since a web page's HTML/images/scripts genuinely need to arrive complete and in order).

Request structure

```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html

(optional body, e.g. for POST)
```

- **Method** (GET, POST, ...), **path**, **HTTP version**, then **headers**, then an optional **body**.

Response structure

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1234

<html>...</html>
```

- **Status line** (version, status code, reason phrase), **headers**, then the **body** (the actual content).

2. Common HTTP Methods

Method	Purpose	Idempotent?
GET	Retrieve a resource	Yes
POST	Submit data (e.g., create a resource)	No
PUT	Replace a resource entirely	Yes
PATCH	Partially update a resource	No (generally)
DELETE	Remove a resource	Yes
HEAD	Like GET, but only headers — no body (useful for checking existence/metadata cheaply)	Yes

Idempotent means: making the same request multiple times has the same effect as making it once. This matters a lot in practice — e.g., it's why browsers warn before resubmitting a POST form on refresh, but not a GET.

3. Status Code Classes

Range	Class	Example
1xx	Informational	100 Continue
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirection	301 Moved Permanently, 302 Found, 304 Not Modified
4xx	Client error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Server error	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

401 vs 403 — a common trip-up: 401 (Unauthorized) actually means "you haven't authenticated" (despite the name); 403 (Forbidden) means "you're authenticated, but not permitted to do this." Interviewers frequently probe this distinction.

4. Statelessness — and how the web fakes state anyway

HTTP is fundamentally stateless — each request is handled completely independently; the server has no built-in memory of any prior request from the same client.

This creates an obvious problem: how does a website know you're logged in across multiple page loads? Two main mechanisms exist:

- **Cookies** — the server sends a `Set-Cookie` header in a response; the browser stores it and automatically re-sends it (via a `Cookie` header) on every subsequent request to that domain. The cookie value is typically an opaque session identifier.
- **Sessions** — the server maintains actual session state (e.g., "user 42 is logged in") server-side, keyed by the session identifier the client's cookie carries — the cookie itself holds no meaningful data, just a lookup key.

Interview soundbite: "HTTP itself has no concept of a 'logged-in user' across requests — every request is a fresh, isolated transaction. Cookies are the bolt-on mechanism that lets a stateless protocol simulate a stateful experience, by having the client carry a small token that lets the server recognize 'this request belongs to the same session as that earlier one.'"

5. Connection Handling: HTTP/1.0 vs 1.1 vs 2 vs HTTPS

- **HTTP/1.0** — opened a **new TCP connection for every single request** (extremely wasteful — each request paid the full 3-way-handshake cost).
 - **HTTP/1.1** — introduced **persistent connections** (`Connection: keep-alive`) — one TCP connection can serve multiple sequential requests, avoiding repeated handshake overhead. Still fundamentally **one request in flight at a time** per connection (head-of-line blocking at the application level), though pipelining was theoretically possible.
 - **HTTP/2** — introduces **multiplexing**: multiple requests/responses can be interleaved over a **single** TCP connection simultaneously, eliminating HTTP/1.1's one-at-a-time limitation. Also adds header compression and server push.
 - **HTTPS** — not a separate protocol, but HTTP layered **on top of TLS/SSL** — TLS handles encryption, authentication (via certificates), and integrity, and HTTP runs unmodified inside that encrypted tunnel.
-

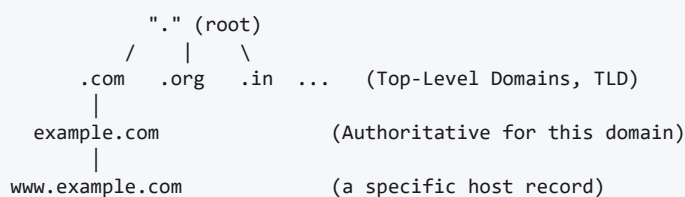
PART 2 — DNS (Domain Name System)

6. The Problem DNS Solves

Humans want to type `www.example.com` ; machines need an IP address to actually route a packet. DNS is the **distributed, hierarchical** system that translates domain names into IP addresses (and back, and handles several other lookup types).

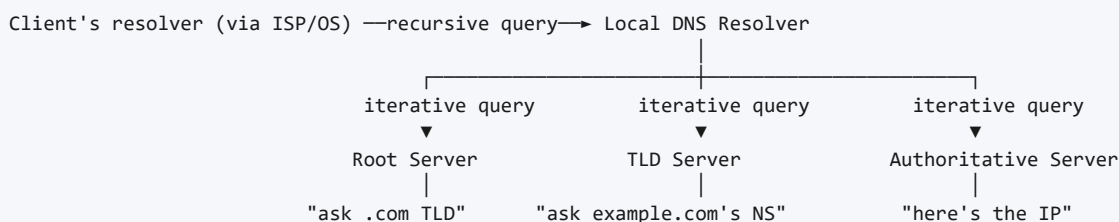
Why distributed and hierarchical, rather than one giant lookup table? No single server could handle query volume for the entire Internet's naming, and no single organization should have unilateral control over the entire namespace — hierarchy lets responsibility (and load) be delegated down a tree, exactly the same "why does a system need structure" reasoning that shows up in OS design and routing.

7. The DNS Hierarchy



- **Root servers** — know only how to direct queries to the correct **TLD** servers (`.com` , `.org` , `.in` , etc.) — a small, well-known set of servers at the very top.
- **TLD servers** — know which **authoritative** server is responsible for each specific domain under that TLD.
- **Authoritative servers** — hold the actual, definitive DNS records for a specific domain (e.g., `example.com` 's own DNS server knows `www.example.com` 's actual IP).

8. The Resolution Process



- **Recursive query** — the client asks its resolver "just give me the final answer" — the resolver does all the work and hands back one complete result.

- **Iterative query** — the resolver, on the client's behalf, walks down the hierarchy itself: asks the root "who handles .com?", asks that TLD server "who's authoritative for example.com?", asks that authoritative server "what's www.example.com's IP?" — each server gives either the final answer or a referral to the next server down, rather than doing the walking itself.
- **Caching** — every resolver along the way **caches** the answer for a duration specified by the record's **TTL (Time To Live)**, so repeated lookups for the same name don't need to repeat this whole walk. This is exactly why DNS changes can take time to "propagate" — cached (now-stale) answers persist at various resolvers until their TTL expires.

9. Common DNS Record Types

Record	Purpose
A	Maps a hostname to an IPv4 address
AAAA	Maps a hostname to an IPv6 address
CNAME	An alias — points one hostname to another hostname (which is then resolved again)
MX	Mail exchange — which mail server handles email for this domain (used by SMTP, see below)
NS	Which name server is authoritative for this domain
TXT	Arbitrary text — commonly used for domain verification, SPF/email-security records

Interview soundbite: "DNS is a classic example of trading a small amount of staleness (via TTL-bounded caching) for a massive reduction in load on authoritative servers — nearly every lookup for a popular domain is served from some resolver's cache, not by walking the hierarchy from the root every single time."

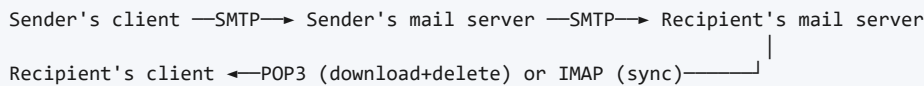
PART 3 — Email Protocols

10. The Three Protocols, and Why There Are Three (Not One)

Sending and receiving email are genuinely different problems, handled by different protocols:

- **SMTP (Simple Mail Transfer Protocol)** — used to **send** mail (client → outgoing server, and server → server as it relays toward the recipient's mail server). SMTP is a **push** protocol.

- **POP3 (Post Office Protocol v3)** — used by a mail client to **retrieve** (pull) mail from a mail server. Classically, POP3 **downloads and deletes** mail from the server, moving it entirely onto the local device.
- **IMAP (Internet Message Access Protocol)** — also used to retrieve mail, but designed to **keep mail synchronized on the server** — read/unread status, folders, and the messages themselves stay server-side, letting the same mailbox be accessed consistently from multiple devices.



Why POP3 still exists at all, given IMAP's clear advantages for multi-device use:

POP3's download-and-delete model means mail isn't left sitting on a potentially storage-constrained server — useful for constrained mailboxes or single-device use, and simpler to implement. But for essentially all modern multi-device usage, **IMAP is the practical default**.

11. MIME — extending email beyond plain ASCII text

The original SMTP was designed for simple 7-bit ASCII text. **MIME (Multipurpose Internet Mail Extensions)** extends this to support attachments, images, non-ASCII character sets, and rich formatting (HTML email) — MIME headers describe the content type and encoding, and SMTP transports the resulting MIME-formatted message without needing to understand its internal structure at all.

Interview Questions With Answers

Q1. Why does HTTP run over TCP rather than UDP?

Answer: Web content (HTML, images, scripts) generally needs to arrive completely and in the correct order for the page to render correctly at all — a dropped or reordered chunk of an HTML file isn't "good enough," unlike, say, a dropped video frame. TCP's reliable, ordered, connection-oriented delivery directly matches this requirement, whereas UDP's best-effort, unordered delivery would require HTTP to reimplement reliability and ordering itself for little benefit in this use case (this is precisely why HTTP/3, built on QUIC over UDP, still re-implements reliability at a different layer rather than doing without it).

Q2. What's the actual difference between a 401 and a 403 status code?

Answer: 401 (Unauthorized) means the request lacks valid authentication credentials at all — the server doesn't know who you are (despite the somewhat misleading name, it's really about authentication, not authorization). 403 (Forbidden) means the server does know who you are (or

doesn't need to), but the authenticated identity simply isn't permitted to perform this specific action or access this specific resource.

Q3. HTTP is described as stateless. How do cookies work around this without changing HTTP's fundamental nature?

Answer: HTTP's statelessness means the protocol itself has no built-in memory of prior requests — this doesn't change with cookies. Cookies work *around* it by having the server hand the client a small opaque token (via `Set-Cookie`), which the client's browser then automatically attaches to every subsequent request to that domain (`Cookie` header). The server uses this token purely as a lookup key into its own server-side session state — HTTP as a protocol still treats every request independently; it's the *application* built on top of HTTP that reconstructs continuity by looking up state keyed by whatever token arrives with each otherwise-independent request.

Q4. What specific limitation of HTTP/1.1 does HTTP/2's multiplexing solve?

Answer: HTTP/1.1, even with persistent connections, generally handles requests one at a time per connection — a slow response to one request blocks subsequent requests on that same connection from completing (head-of-line blocking at the application level), which browsers historically worked around by opening several parallel TCP connections to the same server. HTTP/2's multiplexing allows multiple requests and responses to be interleaved over a *single* TCP connection simultaneously, so one slow response no longer blocks others behind it on the same connection, removing the need for those multiple parallel connections as a workaround.

Q5. Why is DNS designed as a hierarchical, distributed system rather than one central database?

Answer: A single centralized database would need to handle the query volume of the entire Internet's name lookups, representing both a performance bottleneck and a single point of failure, and would require one entity to have unilateral authority over the entire global namespace. The hierarchical design delegates responsibility down a tree (root → TLD → authoritative servers for each domain), spreading both query load and administrative authority — each organization manages its own domain's records independently, and no single server needs to know everything.

Q6. Explain the difference between a recursive and an iterative DNS query.

Answer: In a recursive query, the requester asks a resolver for a complete, final answer, and that resolver takes on the full responsibility of walking the DNS hierarchy (root → TLD → authoritative) itself before returning one final result. In an iterative query, each server queried returns either the final answer or a referral to the next server to ask, and the querier (typically a resolver acting on a client's behalf) is the one doing the walking, step by step, itself. In practice, clients issue recursive queries to their local resolver, and that resolver then issues a series of iterative queries up and down the DNS hierarchy on the client's behalf.

Q7. Why does DNS caching (via TTL) exist, and what's the trade-off it accepts?

Answer: Without caching, every single DNS lookup for even the most popular domains would have to walk the entire hierarchy from a resolver down to an authoritative server, which would be an enormous, unnecessary load on root/TLD/authoritative servers given how repetitive most lookups are (the same popular domains get queried constantly). TTL-bounded caching lets resolvers reuse a previous answer for a bounded time, dramatically cutting this load — the trade-off is staleness: if a domain's IP address changes, resolvers holding a cached answer won't see the update until their cached copy's TTL expires, which is exactly why DNS changes are described as taking time to "propagate."

Q8. Why are there three separate email protocols (SMTP, POP3, IMAP) instead of one unified protocol?

Answer: Sending mail and retrieving mail are fundamentally different operations with different requirements: SMTP is a push protocol optimized for relaying a message from sender to recipient's mail server (potentially hop by hop across multiple servers), while POP3/IMAP are pull protocols optimized for a client fetching mail that's already arrived and is sitting in a mailbox. Within retrieval, POP3 and IMAP further differ because they serve different usage patterns: POP3's simple download-and-delete model suits single-device, storage-constrained use, while IMAP's server-side synchronization model suits accessing the same mailbox consistently from multiple devices — a single protocol trying to serve all of send, simple single-device retrieval, and multi-device sync well would be more complex than three protocols each doing one job well.

Q9. Scenario: A user reports that after their company migrated its website to a new server with a new IP address, some users can access the new site immediately while others still see the old site for hours. What's the most likely explanation, and is anything actually "broken"?

Answer: Nothing is broken — this is expected DNS caching (TTL) behavior. Different users' DNS resolvers cached the domain's old A record at different times, each with its own TTL countdown; users whose cached entry has already expired will perform a fresh lookup and get the new IP immediately, while users whose resolver is still within the old record's TTL window continue being served the stale, cached IP address until that TTL expires and a fresh lookup occurs. This is exactly why DNS best practice recommends lowering a record's TTL *in advance* of a planned IP change (giving existing cached copies time to expire naturally before the change happens), so that when the change actually occurs, resolvers are more likely to already be due for a fresh lookup rather than serving a long-lived stale cache entry.